

PouetPouet — Stack technique

Plateforme collaborative : whiteboards temps-réel, animations de session (vote, timer, sondages), Scrum Poker, Daily standup et roue aléatoire.

Vue d'ensemble

Couche	Technologie
Monorepo	Turborepo + npm workspaces
Backend	Fastify 5 + TypeScript (ES modules)
Realtime	Socket.io 4
ORM	Prisma 6
Base de données	PostgreSQL 16
Cache / pub-sub (optionnel)	Redis 7
Auth	JWT (@fastify/jwt) + bcrypt
Frontend	Next.js 15 (App Router) + React 19
Styling	Tailwind CSS 4
State client	Zustand 5
Validation	Zod (schémas partagés entre routes et sockets)
Runtime	Node ≥ 20

Structure du repo

```
PouetPouet/
├─ apps/
│  └─ api/                # Fastify + Prisma + Socket.io
│     └─ prisma/
│        └─ schema.prisma
│        └─ migrations/
│     └─ src/
│        └─ index.ts      # Bootstrap Fastify + Socket.io
│        └─ lib/          # prisma, io, redis
│        └─ routes/       # REST endpoints (auth, boards, templates, sessions, scrum, daily, wheel)
│        └─ sockets/      # Handlers temps-réel (board, vote, session, scrum, daily)
│  └─ web/                # Next.js 15 App Router
│     └─ src/
│        └─ app/
│           └─ (auth)/    # login, register
│           └─ (app)/     # dashboard, boards, daily, scrum, wheel, equipes, profile
│           └─ join/      # Rejoindre une session via code
│           └─ session/   # UI participant
```

```
|       | components/ # board/, dashboard/, session/, ui/
|       | hooks/      # useBoard, useBoards, useTemplates, useSession, useDaily, useScrum, useWheel
|       | lib/        # api (REST client), socket (Socket.io client)
|       | store/      # Zustand auth store
|-----| packages/
|       |   shared/    # Types TypeScript partagés (user, board, session, activity, socket)
|-----| docker-compose.yml # Postgres + Redis pour dev local
|-----| turbo.json
|-----| package.json      # Workspaces racine
```

Démarrer en local

Prérequis

- Node $\geq$ 20
- Docker (pour PostgreSQL et Redis)
- npm $\geq$ 11

Setup

```
# 1. Installer les dépendances
npm install

# 2. Démarrer Postgres + Redis
docker compose up -d

# 3. Variables d'environnement API
# apps/api/.env
DATABASE_URL=postgresql://pouetpouet:pouetpouet_dev@localhost:5432/pouetpouet
JWT_SECRET=change-me-in-production
FRONTEND_URL=http://localhost:3000

# 4. Variables d'environnement web (optionnel)
# apps/web/.env.local
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_SOCKET_URL=http://localhost:4000

# 5. Appliquer les migrations Prisma
cd apps/api && npx prisma migrate deploy && npx prisma generate

# 6. Lancer tout (depuis la racine)
npm run dev
```

L'API tourne sur `http://localhost:4000` , le web sur `http://localhost:3000` .

Scripts utiles

Commande	Effet
<code>npm run dev</code>	Lance API + web en parallèle (turbo)
<code>npm run build</code>	Build les deux apps

<code>npm run lint</code>	Typecheck via turbo
<code>cd apps/api && npm run db:migrate</code>	Crée une nouvelle migration
<code>cd apps/api && npm run db:studio</code>	Ouvre Prisma Studio

Modèle de données

Domaines principaux

Boards collaboratifs

- `User` — comptes (email, password bcrypt, avatar, bio, theme)
- `Board` — espace de travail (nom, description, cover, max participants, activités, shareToken, templateDraftOf)
- `Card` — cartes du board (TEXT, IMAGE, LINK, SHAPE, DRAW, LABEL) avec position, taille, couleur, lock, groupe, **layer** (1 = fond, 2 = principal, 3 = avant-plan)
- `Frame` — cadres regroupant des cartes
- `CardConnection` — flèches entre cartes
- `BoardField` — champs personnalisés (TEXT, NUMBER, DATE, SELECT)
- `CardFieldValue` — valeurs des champs sur chaque carte
- `BoardShare` — partages par rôle (VIEWER, EDITOR)
- `BoardFavorite` — favoris utilisateur
- `BoardTemplate` — templates (snapshot JSON de cards/frames/connections/fields)
- `BoardVoteSession` + `BoardVote` — sessions de vote sur cartes

Sessions live

- `Session` — session live d'un board (code à 6 caractères)
- `SessionParticipant` — participants (utilisateur connecté ou invité)
- `Activity` + `ActivityResponse` — animations (QUIZ, POLL, WORDCLOUD, BRAINSTORM, QA)

Scrum Poker

- `ScrumRoom`, `ScrumTicket`, `ScrumVote` — estimation collaborative (Story Points / Fibonacci ou Temps) ; `ScrumTicket` porte `estimate` (points) et `estimateTime` (durée)

Daily standup

- `DailyTeam`, `DailyTeamMember` — équipes persistées
- `DailySession`, `DailyParticipant` — sessions de daily avec timer par personne

Roue aléatoire

- `WheelEvent`, `WheelDraw` — tirages pondérés (anti-répétition)

Voir [apps/api/prisma/schema/prisma](https://prisma.io/docs/api/prisma/schema/prisma) pour le détail.

Architecture API

REST + Socket.io

Le backend expose deux surfaces :

REST (Fastify, préfixe `/api/`) pour les opérations non temps-réel : auth, listing, création, configuration. Toutes les routes authentifiées passent par `app.authenticate` (preHandler JWT).

Préfixe	Module
/api/auth	inscription, login, profil, mot de passe, avatar, forgot/reset-password
/api/boards	CRUD boards, partages, favoris, settings, vote sessions
/api/templates	CRUD templates + brouillon (edit-content / save-from-draft / discard-draft)
/api/sessions	Création, clôture, join via code, session active par board (GET /active?boardId=), lookup par id
/api/scrum	Scrum Poker rooms et tickets
/api/daily	Équipes et sessions de daily
/api/wheel	Tirages aléatoires et historique

Socket.io pour le temps-réel (rooms par boardId / sessionId / roomId / dailyId) :

Namespace	Fichier	Événements clés
Board	board.ts	board:join/leave/state, card:*, frame:*, boardfield:*, cardfield:*, timer:*, cards:group/lock, board:presence
Vote	vote.ts	vote:start/cast/uncast/stop/extend, vote:session/cast/uncast/closed
Session	session.ts	session:join/host_join/member_join/rejoin/close, activity:launch/close/respond/launched/closed, session:started/closed/rejoined
Scrum	scrum.ts	scrum:join/vote/reveal/reset/next
Daily	daily.ts	daily:start/next/skip/end + diffusion d'état

L'authentification Socket.io se fait via le JWT en `handshake.auth.token` (vérifié par `app.jwt.verify` au moment du `io.use`). Le `userId` est stocké dans `socket.data.userId`.

Permissions

Chaque handler board appelle `canWrite(socket, boardId)` qui regarde `socket.data.boardRoles[boardId]`. Le rôle est calculé une fois au `board:join` (OWNER si propriétaire, sinon look-up `BoardShare`).

Les VIEWER n'émettent jamais d'événements d'écriture côté UI, mais le backend re-vérifie systématiquement.

Resilience

- Toutes les requêtes Prisma passent par un client unique (`apps/api/src/lib/prisma.ts`).
- Les schémas Zod valident les bodies REST **et** les payloads socket avant écriture en base.
- Les opérations destructives (delete board, reset, discard draft) sont protégées côté API (vérification ownerId) ET côté UI (confirmations).

Architecture frontend

App Router

```
src/app/
├── (auth)/                # Pages sans header
```

		login/	# Connexion
		register/	# Inscription
		forgot-password/	# Demande de réinitialisation par email
		reset-password/	# Formulaire de nouveau mot de passe (token URL)
		(app)/	# Pages avec layout authentifié (header + nav)
		layout.tsx	# Redirige vers /login si pas de token, gère le dark mode
		join/[token]/	# Landing pour rejoindre un board partagé
		session/[code]/	# UI participant pour une session live

Hooks métier

Chaque domaine a un hook qui combine REST (état initial) + Socket.io (updates) :

- `useBoard(boardId)` — état complet du board avec abonnements socket pour cards/frames/connections/fields/timer/vote/presence ; expose toutes les actions (drag, resize, recolor, lock, group, undo/redo, pasteCards)
- `useBoards()` — listing des boards de l'utilisateur, favoris, création, suppression
- `useTemplates()` — CRUD templates + workflow brouillon
- `useSession(boardId)` — session live côté animateur (hôte) ; reconnexion automatique via `localStorage`
- `useParticipantSession(code)` — session côté participant invité (anonyme) ; reconnexion via `sessionStorage`
- `useBoardSession(boardId, userId, isOwner)` — participation d'un membre authentifié depuis la page board ; écoute `session:started` et `activity:launched`
- `useScrum(roomId)` — Scrum Poker côté hôte ; reconnexion automatique
- `useScrumParticipant()` — Scrum Poker côté participant ; reconnexion automatique via `sessionStorage`
- `useDaily` , `useWheel` — pendants pour les autres modules

State global

Seul l'authentification utilise Zustand (`apps/web/src/store/auth.ts`) — le token JWT est persisté en `localStorage` et synchronisé sur l'objet `user` . Tout le reste est local au composant via les hooks.

Conventions

- Pas de CSS modules ni styled-components — tout en Tailwind avec quelques styles inline pour les éléments dynamiques (position d'une carte, couleurs sélectionnées).
- Les modales sont positionnées en `fixed inset-0` avec backdrop blur.
- Les éléments contextuels (presence dropdown, timer picker) utilisent un portail via `position: fixed` calculé depuis le rect de l'élément déclencheur, pour s'extraire des conteneurs `overflow: hidden` .
- L'undo/redo est local au client (pile de 30 entrées max) avec callbacks `undo / redo` symétriques — les opérations sont rejouées via les mêmes événements socket que l'action originale.

Conventions de code

- **TypeScript strict** partout, modules ES (`"type": "module"` sur l'API)
- **Validation Zod** systématique côté backend pour les inputs externes
- **Migrations Prisma** créées manuellement quand le schéma est sensible, ou via `prisma migrate dev` pour les modifications simples
- **Pas de commentaires** sauf justification (workaround documenté, invariant non évident)
- **i18n** : interface en français, code et commits en anglais

Déploiement

CI/CD

Le pipeline est déclenché automatiquement sur push vers `master` :

- Build Docker de l'API → déployé sur **GCP Cloud Run**
- Build Next.js → déployé séparément

Ne pas pousser sur `master` sans passer par `develop` (les merges `develop`→`master` déclenchent le déploiement automatique).

Build local

```
npm run build
# Génère apps/api/dist + apps/web/.next
```

Prérequis infra

- PostgreSQL accessible via `DATABASE_URL`
- Un seul process Node pour `apps/api` (Socket.io garde l'état en mémoire pour la présence ; pour scaler horizontalement il faudrait activer l'adapter Redis de Socket.io — la dépendance `ioredis` est déjà présente)
- Un host Next.js (Vercel, Node serveur, etc.)
- Le `FRONTEND_URL` côté API doit matcher l'origin du web pour CORS et Socket.io
- Variables d'environnement supplémentaires pour le mailer (réinitialisation de mot de passe + vérification email) : `SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASS` , `SMTP_FROM`

Voir aussi

- [FEATURES.md](#) — Guide utilisateur de toutes les fonctionnalités disponibles
- [ideatank.txt](#) — Idées et améliorations à venir